Evidence-conditioned correlated matching for surface codes under circuit-level noise

Jimmy Carvalho
(Dated: October 14, 2026)

Matching decoders turn each error mechanism of a circuit-level detector error model into graph edges and give every edge the mechanism's full probability, even when the syndrome rules the mechanism out. We show that a local, syndrome-dependent correction to the edge weights brings correlated minimum-weight perfect matching to, or beyond, the accuracy of belief-matching. The rule is that a multi-edge mechanism contributes probability to its edges only if each of its edges touches a detection event. It costs $O(\#\text{detection events})$ per shot, and we combine it with the standard two-pass correlated reweighting. For the rotated surface-code memory under SI1000 noise at distances 5, 7 and 9 and physical error rates 0.2–0.5%, the resulting decoder makes 7–17% fewer logical errors than the correlated mode of PyMatching 2. The advantage grows with distance and holds on circuits not used during development: on Stim's stock circuit with uniform depolarizing noise ($d \leq 9$) and under SI1000 at $d = 11$ the reduction is 11–33%. In paired tests it is statistically indistinguishable from belief-matching at $d = 5, 7$ and better at $d = 9$, while running at 4.6–7.2× the time of uncorrelated PyMatching (1.7–2.2× its correlated mode). The decoder is open source, sinter-compatible and built on PyMatching's sparse blossom.

I. INTRODUCTION

Minimum-weight perfect matching (MWPM) is the standard decoder for the surface code, and sparse blossom [1] makes it fast enough for real-time use. MWPM is not optimal under circuit-level noise, because many error mechanisms flip more than two detectors. A Y error on a data qubit, for example, flips both X-type and Z-type detectors. The detector error model (DEM) decomposes such a mechanism into graph edges, and the decoder then treats those edges as independent.

Correlated matching [2] recovers part of the lost information. It decodes once, lowers the weights of edges that share a mechanism with the chosen edges, and decodes again. PyMatching ships this as `enable_correlations` since version 2.3. Belief-matching [3] goes further: it sets the edge weights from belief-propagation (BP) marginals over the full hypergraph, at roughly 10^3 times the cost.

Correlated matching uses only *positive* evidence: a mechanism becomes more likely when one of its edges is matched. It never uses *negative* evidence, namely that a mechanism whose other part shows no syndrome practically did not happen. Yet every edge in the matching graph carries the full probability of every mechanism containing it. Paths through edges whose correlated partners are silent therefore look cheaper than they are. This paper isolates that effect and corrects it locally.

II. METHOD

a. Graph construction. We merge DEM instructions by symptom, the set of detectors flipped. Each merged mechanism h has probability p_h and a decomposition $E(h)$ into edges. Stim may decompose instructions with the same symptom differently: it splits Y-type errors into X and Z parts even when the symptom is a single edge. When every decomposition is applied separately, as PyMatching does, probability leaks onto edges the mechanism never flips alone, often boundary edges. We instead keep, for each symptom, the offered decomposition with the fewest components. The edge prior is the independent (XOR) combination $q_e = \bigoplus_{h \ni e} p_h$.

b. Evidence gate. Let D be the set of detection events. An edge $e = (u, v)$ is *touched* if $u \in D$ or $v \in D$. A multi-edge mechanism h ($|E(h)| \geq 2$) is *active* if every edge of $E(h)$ is touched. The per-shot edge probability is

$$q_e(D) = \Big(\bigoplus_{h \ni e,\ |E(h)|=1} p_h \Big) \oplus \Big(\bigoplus_{h \ni e,\ h \text{ active}} p_h \Big). \quad (1)$$

An inactive multi-edge mechanism would have to occur together with a second error that cancels its symptom exactly, a higher-order event, so it is dropped. Active mechanisms are found by walking the edges incident to detection events, so the gate costs $O(|D|)$.

c. Two passes. We match with weights $w_e = \log \frac{1-q_e(D)}{q_e(D)}$ and trace the matched paths. Each edge e_2 that shares a mechanism with an edge e_1 on those paths then gets $w_{e_2} \leftarrow \min(w_{e_2}, \log \frac{1-r}{r})$, with $r = \min(\frac{1}{2}, P(e_1 \wedge e_2)/P(e_1))$. This is PyMatching's correlated rule, with joint probabilities precomputed from the DEM. We then match again. All weight changes are written in place into the sparse-blossom graphs and undone after the shot.

III. BENCHMARK

We use the rotated surface-code memory experiment (Z basis, d rounds) from Stim [4]. Each CX is compiled to H·CZ·H, and we apply SI1000 noise [5]: two-qubit depolarizing p after CZ, $p/10$ after single-qubit gates and on idle qubits, an extra $2p$ on qubits idling during measurement or reset, $2p$ reset flips and $5p$ measurement flips. Logical error rates come from sinter with 10^6 shots per point (Fig. 1). Because sinter samples each decoder inde-

TABLE I. Paired comparison with correlated PyMatching (v2.4.0) on identical shots. Ratio = evmatch errors / correlated errors, $\pm 2\sigma$ from the discordant shots. Time is relative to uncorrelated PyMatching.

d	p	shots	ratio	McNemar p	time
5	0.3%	400k	0.924 ± 0.017	5×10^{-20}	4.9×
5	0.5%	400k	0.934 ± 0.008	2×10^{-61}	4.6×
7	0.3%	400k	0.886 ± 0.026	9×10^{-19}	5.1×
7	0.5%	400k	0.920 ± 0.010	4×10^{-62}	4.8×
9	0.2%	2M	0.859 ± 0.051	3×10^{-8}	6.6×
9	0.3%	400k	0.835 ± 0.040	2×10^{-16}	5.9×
9	0.5%	400k	0.887 ± 0.011	5×10^{-97}	4.8×

TABLE II. Decoding time per shot (μs, single thread, same pre-sampled 2×10^4 shots, best of 3) and the ratio of evmatch to uncorrelated PyMatching.

d	p	PyMatching	correlated	evmatch	ratio
5	0.1%	0.9	2.6	4.8	5.2
5	0.2%	2.1	5.6	10.3	4.9
5	0.3%	3.4	9.6	16.5	4.9
5	0.4%	4.7	12.6	23.8	5.0
5	0.5%	6.4	16.4	29.5	4.6
7	0.1%	2.8	8.3	16.2	5.7
7	0.2%	6.3	17.5	35.9	5.7
7	0.3%	10.6	32.4	54.5	5.1
7	0.4%	16.9	43.5	80.4	4.8
7	0.5%	22.1	56.9	106.8	4.8
9	0.1%	6.2	20.7	44.7	7.2
9	0.2%	14.7	46.3	97.2	6.6
9	0.3%	27.1	79.3	159.6	5.9
9	0.4%	40.8	119.8	235.6	5.8
9	0.5%	59.5	166.1	285.0	4.8

pendently, differences of 10% are within noise at the low-count points. We therefore also decode identical shots with both decoders (paired) and count the discordant shots: b, where only the reference fails, and c, where only the new decoder fails. The error ratio is $(n_{\text{ref}} - b + c)/n_{\text{ref}}$ with standard error $\sqrt{b+c}/n_{\text{ref}}$, and McNemar's exact test gives the p-value. Times are single-threaded on the same pre-sampled shots.

IV. RESULTS

Table I and Fig. 1 summarize the results. The reduction relative to correlated matching grows with distance, from 7% at $d = 5$ to 11–17% at $d = 9$. Over the full grid, $p \in \{0.1, \ldots, 0.5\}\%$ (Table II), evmatch costs 4.6–7.2× uncorrelated PyMatching and 1.7–2.2× correlated PyMatching (itself 2.5–3.3×); the worst case is $d = 9$, $p = 0.1\%$, where the fixed per-shot costs of path tracing dominate.

a. Ablation. Table III separates the contributions at $d = 9$, $p = 0.3\%$. The fewest-components decomposition alone gives 2%. Dropping hyperedges from the base graph without any evidence gate gives a further 4%; this is like the first pass of the correlated decoder inside Harmony [6]. The evidence gate contributes the remaining 11%. Applying the gate to the second pass only recovers most, but not all, of that 11%. Using the gate without the correlated pass is *worse* than correlated matching: the positive and the negative evidence are complementary.

TABLE III. Ablation, paired, $d = 9$, $p = 0.3\%$, 400k shots.

decoder	errors
PyMatching, correlated	1210
+ fewest-components decomposition	1184
+ hyperedge-free base graph, no gate	1141
+ evidence gate, second pass only	1054
+ evidence gate, both passes (evmatch)	1010
evidence gate, single pass, no correlations	1965

b. Belief-matching. Against the reference `beliefmatching` package (20 BP iterations), on identical shots, the error counts were 1096 vs 1132 ($d = 5$, $p = 0.3\%$, 10^5 shots) and 4761 vs 4696 ($p = 0.5\%$), and 290 vs 309 and 2148 vs 2180 at $d = 7$ (5×10^4 shots). None of these differences is significant ($p > 0.15$). At $d = 9$, $p = 0.3\%$, evmatch made 70 errors against 110 for belief-matching (2.5×10^4 shots, $p = 9 \times 10^{-6}$); there the reference implementation was also worse than correlated matching (85 errors), with or without its shortcut of returning BP's solution when BP converges, with 30 instead of 20 BP iterations (104 errors), and in an independent implementation of belief-matching with our decomposition and log-odds weights (107 errors; 972 at $p = 0.5\%$), with min-sum BP (104) and with a serial BP schedule (103). Belief-matching with parallel BP therefore does not dominate correlated matching on this circuit, while evmatch matches or beats both at every point we measured. At $d = 9$, $p = 0.5\%$, the counts were 937 vs 971 ($p = 0.23$).

c. Other circuits. All tuning was done on the SI1000 circuit above. Table IV repeats the paired comparison on circuits that were never used during development: Stim's stock `rotated_memory_z` and `rotated_memory_x` with plain CX gates and uniform depolarizing circuit noise (probability p after every Clifford, on data qubits before each round, before measurements and after resets), and the SI1000 circuit at $d = 11$. Every point favours evmatch, the reduction grows with distance, and the cost stays at 1.7–2.0× correlated PyMatching.

V. RELATED WORK

Positive-only correlated reweighting appears in Fowler's original method [2], its pipelined version [7], PyMatching 2.3, Tesseract's multi-pass mode, and the alternating X/Z schemes of Refs. [8, 9]. Detection-event-driven local reweighting also appears in Google's

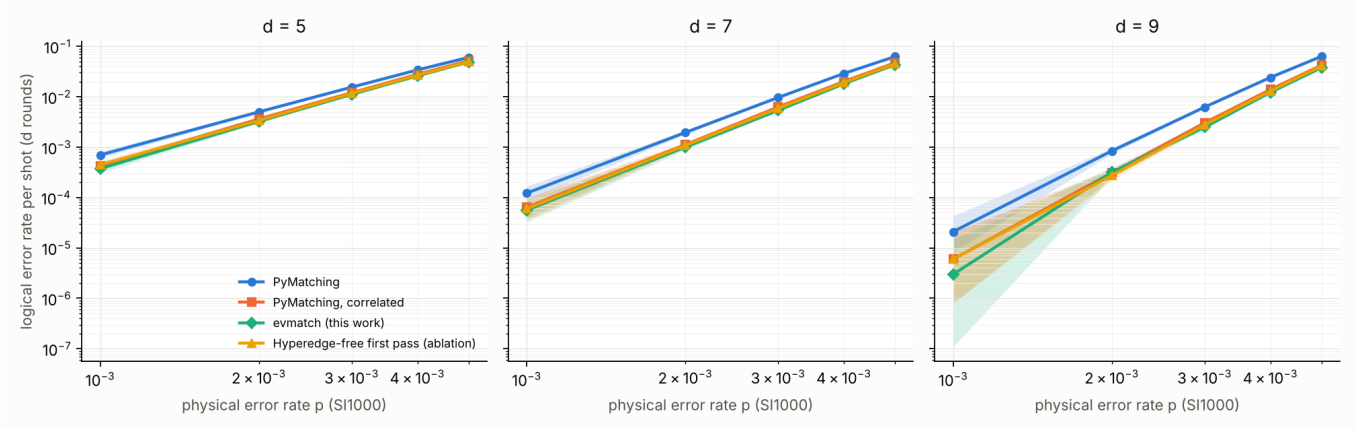


FIG. 1. Logical error rate per shot (d rounds) under SI1000 noise, 10^6 shots per point (sinter); bands are likelihood intervals (max-likelihood factor 1000).

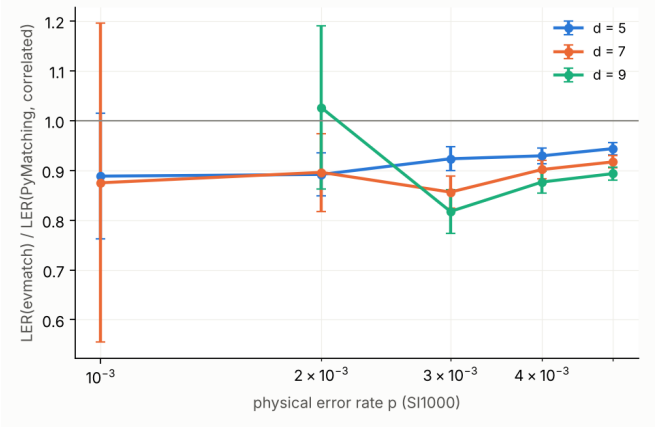


FIG. 2. Ratio of logical error rates, evmatch over correlated PyMatching, from the independent sinter samples of Fig. 1 (points with at least 50 errors; bars $\pm 2\sigma$). The paired comparisons of Table I are more precise.

real-time decoder ("preweight correlations") [10], in Paler and Fowler's virtual pre-matching [7], and in the split-fault rule of Ref. [11]. All of these only *lower* weights. The DGR decoder [12] also raises weights, but it conditions on a previous matching, not on the syndrome, and learns correlations across shots. Belief-matching [3] captures negative evidence through global BP. Ensembles (Harmony [6], Libra [13]), neural decoders [14] and search-based decoders [15] are more accurate but cost orders of magnitude more or are not open. Decoder switching [16] sends only low-confidence shots to belief-matching and is complementary to our approach.

TABLE IV. Paired comparison with correlated PyMatching on circuits not used during development (errors on identical shots; $\pm 2\sigma$ from the discordant shots).

circuit	d, p	shots	PM	corr.	evmatch	ratio
stock, Z	5, 0.5%	200k	2202		1962	0.89 ± 0.03
stock, Z	5, 0.3%	400k	955		852	0.89 ± 0.04
stock, Z	7, 0.5%	200k	1182		1016	0.86 ± 0.04
stock, Z	7, 0.3%	400k	271		227	0.84 ± 0.09
stock, Z	9, 0.5%	200k	656		530	0.81 ± 0.06
stock, Z	9, 0.3%	400k	82		55	0.67 ± 0.15
stock, X	7, 0.5%	200k	1324		1088	0.82 ± 0.04
SI1000	11, 0.3%	400k	546		424	0.78 ± 0.06
SI1000	11, 0.5%	200k	7508		6489	0.86 ± 0.02

VI. DISCUSSION

The evidence gate is a crude, local stand-in for the BP posterior: it keeps the part of the conditioning that matters to a matching decoder, namely whether a correlated partner is visible in the syndrome. It drops the part that hurts matching, which is penalizing the interior of error chains whose detectors fire twice. It needs nothing beyond the DEM, adds no parameters, and fits into existing correlated-matching implementations as a different base graph plus a per-shot restore step. Open questions include graded rather than binary evidence, more than two passes, other codes (colour codes, where most mechanisms are hyperedges), and hardware data.

a. Acknowledgments. The software, the experiments and a first draft of this manuscript were produced with the assistance of an AI system (Claude, Anthropic).

b. Code and data. `https://github.com/JimmyCarvalho/evmatch` (Apache-2.0), including the circuits, all raw sinter statistics and the scripts that produce every number in this paper.

[1] O. Higgott and C. Gidney, Sparse Blossom: correcting a million errors per core second with minimum-weight matching, Quantum **9**, 1600 (2025), arXiv:2303.15933.

[2] A. G. Fowler, Optimal complexity correction of correlated errors in the surface code, arXiv:1310.0863 (2013).

[3] O. Higgott, T. C. Bohdanowicz, A. Kubica, S. T. Flammia and E. T. Campbell, Improved decoding of circuit noise and fragile boundaries of tailored surface codes, Phys. Rev. X **13**, 031007 (2023).

[4] C. Gidney, Stim: a fast stabilizer circuit simulator, Quantum **5**, 497 (2021).

[5] C. Gidney, M. Newman and M. McEwen, Benchmarking the planar honeycomb code, Quantum **6**, 813 (2022).

[6] N. Shutty, M. Newman and B. Villalonga, Efficient near-optimal decoding of the surface code through ensembling, arXiv:2401.12434 (2024).

[7] A. Paler and A. G. Fowler, Pipelined correlated minimum weight perfect matching of the surface code, Quantum **7**, 1205 (2023).

[8] N. Delfosse and J.-P. Tillich, A decoding algorithm for CSS codes using the X/Z correlations, arXiv:1401.6975 (2014).

[9] Y. Yuan and C.-C. Lu, A modified MWPM decoding algorithm for quantum surface codes over depolarizing channels, arXiv:2202.11239 (2022).

[10] Google Quantum AI and collaborators, Quantum error correction below the surface code threshold, Nature **638**, 920 (2025).

[11] L. Grans-Samuelsson *et al.*, Improved pairwise measurement-based surface code, Quantum **8**, 1429 (2024), arXiv:2310.12981.

[12] H. Wang, P. Liu, Y. Liu, J. Gu, J. Baker, F. T. Chong and S. Han, DGR: Tackling drifted and correlated noise in quantum error correction via decoding graph re-weighting, arXiv:2311.16214 (2023).

[13] C. Jones, Improved accuracy for decoding surface codes with matching synthesis, arXiv:2408.12135 (2024).

[14] J. Bausch *et al.*, Learning high-accuracy error decoding for quantum processors, Nature **635**, 834 (2024).

[15] L. A. Beni, O. Higgott and N. Shutty, Tesseract: a search-based decoder for quantum error correction, arXiv:2503.10988 (2025).

[16] R. Toshio, K. Kishi, J. Fujisaki, H. Oshima, S. Sato and K. Fujii, Decoder switching: breaking the speed-accuracy tradeoff in real-time quantum error correction, arXiv:2510.25222 (2025).